

How Far Can 16 GB Go? Characterizing Memory-Efficient LLM Fine-Tuning on a Single Consumer Apple Silicon Mac

Yuanyuan Ma

September 2026

Abstract

We map the fine-tuning feasibility boundary of a 16 GiB Apple M4 Mac mini with a preregistered benchmark: 118 supervised runs over the Qwen3 family (0.6B–14B), BF16 LoRA vs. 4-bit QLoRA, context lengths 512–8192, LoRA ranks 4–32, and micro-batches 1–8, each with full provenance (git commit, model revision, per-second swap sampling, SHA-256 manifests). We find: (1) 4-bit QLoRA cuts peak training memory to $0.54\text{--}0.73\times$ of BF16, and 8B is the highest scale whose preregistered seed set completed three-of-three within one favorable residency window; step-time ratios of $1.10\text{--}1.23$ stay inside the preregistered $\pm 25\%$ equivalence margin at both fully paired scales (the third scale contributes a single cross-window pair, reported as such). (2) The transition from reproducibly trainable configurations to system-state-dependent boundary behavior is directly observable: the same 4B BF16 or 14B 4-bit configuration that trains at zero or low swap residency stalls at 70% completion or makes zero progress when several GiB of swap are resident. We report 14B as an explicit boundary case rather than a completed formal cell. (3) Peak memory grows *sublinearly* with model size over the measured range (log-log slope 0.57 for 4-bit, 95% CI $[0.37, 0.78]$), consistent with fixed-overhead dilution, while step time grows approximately linearly (slope 1.00; absolute step times are window-specific and only within-window ratios are compared). (4) Context length, not model scale, is the binding constraint: 2048-token contexts complete but push the allocator high-water mark to 23.9 GiB (beyond physical RAM, carried by macOS paging), and 4096-token contexts are killed before their first step. (5) Micro-batch scaling fails outright at batch 8 (external SIGKILL, with a kernel jetsam record confirming one kill, while system-wide swap usage nears 20 GiB) and *reduces* throughput at every trainable size. Every number in this paper is generated programmatically from immutable raw results; nine preregistration deviations, including one

post-result audit-rule revision, are reported in full.

1 Introduction

Consumer Apple Silicon machines ship with a single 16 GiB unified-memory pool that the GPU shares with the OS. Whether, and under what conditions, such a machine can fine-tune modern LLMs is usually answered anecdotally. This paper answers it with a preregistered, failure-inclusive benchmark: every configuration in a frozen matrix was run under an experiment supervisor that recorded provenance, per-second swap sampling, and complete logs. Every completed or failed run is retained as evidence.

Our contributions: (i) a preregistered feasibility map for Qwen3 0.6B–14B under BF16 LoRA, 4-bit QLoRA, and (exploratory) full fine-tuning on a single M4/16 GiB machine, with 8B 4-bit identified as the highest configuration whose preregistered seed set completed three-of-three in one favorable residency window and 14B reported as an explicit system-state-dependent boundary case; (ii) quantified memory/time trade-offs with frozen equivalence margins; (iii) context-length and micro-batch boundaries expressed as intervals with system-state confounders documented; (iv) an audit showing that TRAINABLE is a joint property of the model and the machine’s memory-residency state, not of the model alone; and (v) a full reproducibility chain from raw immutable results to every number in this paper, including nine logged deviations from preregistration.

2 Background and Related Work

2.1 Apple Silicon and Unified Memory

Apple Silicon system-on-chips (SoCs) present a single physical memory pool shared by CPU and GPU.

`mlx` [11] is an array framework that targets this architecture directly; `mlx-lm` [16] provides LLM fine-tuning on top of it. Prior profiling of Apple Silicon training reports substantial gaps to NVIDIA GPUs and identifies page faults and kernel-launch overhead as system-level contributors [8]. Formal MLX benchmarking has centered on inference latency for encoder models [1]; training-side MLX reports exist as application case studies [3]; consumer-device LLM stacks beyond MLX include `llama.cpp` [10], and a recent comparative study benchmarks five local inference runtimes on Apple Silicon [17].

2.2 MLX

The MLX framework exposes Metal-backed computation with lazy evaluation and a composable optimizer API [11]. Our measurement validation (Sec. 3.6) quantifies why explicit synchronization is required for per-step timing under lazy execution.

2.3 LoRA and QLoRA

LoRA [13] parameterizes weight updates with low-rank adapters; QLoRA [6] backpropagates through 4-bit quantized frozen weights, following the broader line of LLM quantization work [5]. Both reduce the memory footprint relative to full fine-tuning [12, 4]; GaLore [19] targets the same budget from the optimizer-state side, LOMO [15] from fused low-memory updates, and general-purpose fine-tuning frameworks such as LlamaFactory [20] unify these methods behind one interface. QLoRA’s “paged optimizers” manage GPU memory spikes with CUDA unified-memory paging—a framework-controlled mechanism that has no equivalent in our setting, where macOS owns paging globally and the training process cannot control it (Sec. 3.5). These methods are largely evaluated on data-center GPUs; how they behave inside a 16 GB unified-memory budget is the subject of this paper. On-device fine-tuning on phones and similarly constrained hardware is an adjacent line of work [9], focused on framework engineering rather than feasibility boundaries.

2.4 Resource-Constrained LLM Fine-Tuning

Resource-constrained fine-tuning has been studied mainly on GPUs and mobile devices: memory-efficiency methods such as GaLore [19] and QLoRA [6] report feasibility on GPU hardware where CPU and GPU memory are separate, and on-device frameworks [9] engineer around phone memory budgets; application studies do report Apple Silicon + MLX LoRA runs [3] but without characterizing the feasibility boundary. To our knowledge, that boundary

on a fixed consumer unified-memory budget has not been mapped by a preregistered benchmark: which model scales, precisions, and context lengths admit *completed* training steps, and at what cost? We found no published quantitative boundary for this class of hardware to compare against—the closest works above report feasibility impressions or application outcomes, not boundary measurements—so this paper establishes the reference numbers rather than refining existing ones. Unified memory changes the failure mode relative to GPUs (page thrashing rather than allocator exceptions), which motivates our measurement design.

3 Methodology

3.1 Research Questions

We study:

- **RQ1** How do memory, step time, and throughput of BF16 LoRA and 4-bit QLoRA scale with model size?
- **RQ2** How far does 4-bit quantization extend the TRAINABLE boundary?
- **RQ3** Where is the context-length boundary?
- **RQ4** Does the memory saving of quantization come with a measurable training-time penalty?
- **RQ5** How large is the gap between “training can start” and PRACTICAL fine-tuning?

3.2 Hardware and Software

All experiments run on a single Apple Mac mini with an Apple M4 SoC, 16 GiB unified memory, and macOS 26.5; the exact preflight-recorded environment is in Table 1. Software was locked with `uv`: Python 3.13.11, `mlx` 0.32.2, `mlx-lm` 0.31.3. The M4 generation context is from Apple’s release material [2].

3.3 Models

We use the Qwen3 family [18]: instruction-tuned chat models (not base models) at 0.6B/1.7B/4B in BF16 and 0.6B–14B as pre-quantized 4-bit `mlx-community` conversions; every repository is pinned to a Hub revision and byte-verified (Table 1).

3.4 Dataset

The formal benchmark uses a frozen subset of `ultra-chat.200k` [7] (revision 8049631c): 2,048 training and 32 validation conversations sampled with seed 42 and

Table 1: Hardware, software, and model revisions (provenance from raw experiment records).

Hardware	
Chip	Apple M4
Memory	16 GiB unified
CPU cores	10-core (4P+6E, 10 logical)
OS	macOS 26.5 (build 25F71)
Python	3.13.11 (uv-managed venv)
MLX / mlx-lm	0.32.2 / 0.31.3
Training data	ultrachat_200k@8049631c, frozen 2048/32 subset
Qwen3 models	
	resolved revision (8-char prefix)
Qwen/0.6B	c1899de2...
0.6B-4bit	73e3e38d...
1.7B-4bit	3b1b1768...
Qwen/4B	1cfa9a72...
14B-4bit	a4d9b2df...
Qwen/1.7B	70d244cc...
4B-4bit	4dcb3d10...
8B-4bit	545dc425...

formatted with the Qwen3 chat template; loss is a standard language-model (LM) loss over all tokens. The manifest and SHA-256 sums ship with the repository. Synthetic long-context workloads are used only for context-scaling systems analysis, never for quality claims.

3.5 Fine-Tuning Methods

BF16 LoRA (rank 8, $\alpha=16$, dropout 0, targets q/k/v/o.proj), 4-bit QLoRA (same adapters, group size 64), and, for the smallest model only, full fine-tuning. Optimizer AdamW [14], lr 10^{-4} , constant schedule, micro-batch 1.

QLoRA semantics on MLX Our “4-bit QLoRA” is the `mlx-lm` path: LoRA adapters over a *pre-quantized* 4-bit base with group-wise affine quantization (group size 64), trained by regular AdamW. It differs from the original QLoRA recipe [6] in three ways: the 4-bit format is group-wise affine, not NF4; there is no double quantization of the quantization constants; and there is no paged optimizer—on macOS the OS owns paging globally, so the memory-spike management that QLoRA performs in-framework on GPUs is neither available nor needed in the same form. Numbers in this paper are therefore comparable to the QLoRA *line* of work (low-rank adapters over frozen quantized weights) but not configuration-identical to Dettmers et al.’s setup.

3.6 Measurement Methodology

Every run is launched by an Experiment Supervisor (Figure 1) that records git provenance, hard-

ware, memory/swap state (before, after, and 1 Hz sampling), full stdout/stderr, per-step timings, and an atomic SHA-256 manifest; failures are first-class results. Step timing includes forward, backward, optimizer update, and an explicit `mx.eval` synchronization; we measured that omitting this synchronization under-reports step time by three orders of magnitude under MLX’s lazy execution. Peak memory is the MLX allocator’s Metal-buffer high-water mark (`get_peak_memory`); it is not process resident set size (RSS), and on unified memory it may exceed physical RAM, at which point macOS paging carries the excess. The before/after snapshots additionally record the full `vm_stat` counter set (swap-ins/outs, compressor occupancy, compressions/decompressions); the 1 Hz trajectory samples system-wide swap only. Paging intensity per run is computed from the `vm_stat` snapshots as swap-in bytes per completed step (the D4 stratification of Table 8). System-level counters include background processes, sampled peaks may miss transients between samples, and the reported peak is the maximum *observed* value—the validation details and these limitations are in the repository (`docs/measurement-validation.md`).

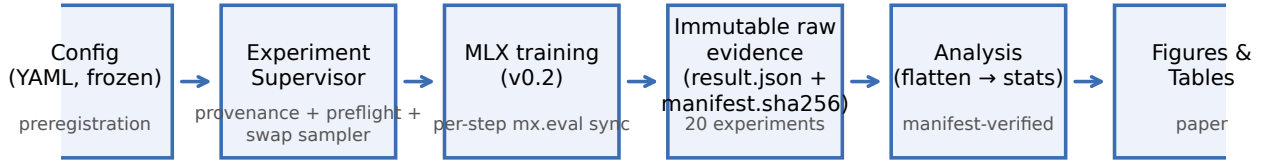
3.7 Operational Definitions

LOADABLE: the training process loads the model weights. **TRAINABLE:** the run completes every requested optimizer step with `terminal_state=success` and no kill signal. **PRACTICAL:** TRAINABLE and (P1) sampled peak system swap does not exceed the pre-run level by more than 4 GiB, and (P2) median step time ≤ 10 s. **EFFICIENT:** PRACTICAL with throughput ≥ 100 loss-bearing tokens/s. All thresholds were frozen before any formal run (commit `3dc33d2`, 2026-09-12; `research/preregistration.md`); sensitivity to $\pm 2\times$ threshold changes is reported in Table 9, where the system-wide P1 criterion is shown to be confounded by background-process swap noise (Sec. 6.1). **SIGKILL** terminations are never reclassified as OOM without direct kernel evidence.

3.8 Reproducibility

Formal configurations, the dataset freeze, the analysis pipeline, and this paper’s tables and figures are generated from raw results by committed scripts; no number is hand-typed. Appendix A lists reproduction commands.

Benchmark pipeline



every number in the paper traces back to an immutable raw result

Figure 1: Benchmark pipeline: preregistered configs are executed by the experiment supervisor, which freezes every run as immutable raw evidence with provenance and a SHA-256 manifest; analysis, tables, and figures are regenerated from raw results by committed scripts only.

4 Results

4.1 Model-Scale Feasibility

Table 2 and Figure 2 summarize the preregistered matrix at ctx512. All BF16 LoRA and 4-bit QLoRA cells through 1.7B complete their three seeds, and 4-bit completes at 4B and 8B as well (8B: three seeds, 1,643–1,693s per 100-step run, 9.19 GiB peak). The three 8B successes all fall in one 3.6–4.8 GiB-residency window on 2026-09-15; an earlier first attempt at 16.4 GiB residency timed out and is retained as system-state evidence (D7)—the same regime dependence the boundary cells exhibit, read here in the favorable direction. Three-of-three seeds also implies a wide success-rate interval (Clopper–Pearson 95% CI lower bound 0.29), so we read “reproducibly trainable” as *repeatable within one observed window*. 8B 4-bit is thus the highest configuration with a completed preregistered seed set in this benchmark. The 14B 4-bit cell is deliberately reported as a system-state-dependent boundary case rather than a completed or failed formal cell (deviation D8; Sec. 4.8). The 4B BF16 cell shows regime dependence (D1): it completed in 451s at zero residency but produced zero completed steps in 27 minutes at 6.1–6.3 GiB residency. 8B BF16 was preregistered as a probe only and timed out; 14B BF16 was declared out-of-budget without observation. Full fine-tuning of 0.6B completes but diverges at the frozen lr 10^{-4} (final validation loss 4.44 vs. 1.93 for 4-bit QLoRA), a negative quality result we retain as-is.

4.2 Memory Scaling

Figure 3 shows peak MLX-allocator memory versus logical parameters. Memory grows *sublinearly* over

the measured range: log-log slope 0.57 ± 0.05 (95% CI $[0.37, 0.78]$, $n=4$) for 4-bit QLoRA, an interval that excludes the parameter-proportional value of 1. The BF16 series has three points; its slope 0.71 ± 0.07 (CI $[-0.22, 1.65]$) carries no inferential weight at that sample size. A power law and the affine model peak $\approx w\text{-params} + c$ fit the 4-bit points comparably (raw-space ΔAIC 3.2 in favor of the power law, both $R^2 > 0.98$), and the affine fit quantifies the mechanism directly: a fixed non-weight component of ~ 1.8 GiB (intercept) plus ~ 0.93 GiB per B parameters. Fixed-overhead dilution is visible in the split: at 0.6B the 4-bit weights occupy only ~ 0.31 GiB of a 2.13 GiB peak, while at 8B the split is 4.29 GiB weights inside a 9.19 GiB peak. We report audit H4 (approximate linearity) as unsupported over this range, with an explicit caveat: sublinearity is a bounded-range observation. The affine mechanism approaches slope 1 as parameters grow, and the 8B allocator peak of 9.19 GiB says nothing about 14B without new measurements. Within this range, memory ceilings bind later than parameter-proportional estimates suggest.

4.3 Training-Time Scaling

Step time scales approximately linearly, not sublinearly, with model size (Figure 4; log-log slope 1.00 ± 0.09 , 95% CI $[0.62, 1.37]$, $R^2=0.985$, 4-bit series, $n=4$; audit H5: partially supported, approximately-linear band). Under the preregistered paging stratification (D4) the picture is mixed rather than uniformly paging-confounded (Table 8): the 0.6B/1.7B 4-bit groups and the 0.6B BF16 group are Tier-A (<50 MB swap-in per step), while the 4B and 8B groups are Tier-B (100–122 and 732–848 MB/step respectively). An earlier draft, and the audit behind it, labeled every run

Table 2: Formal benchmark at ctx=512, b1-ga1, rank 8, lr 1e-4, 100 steps, preregistered seeds {42,123,2026} (mean $\pm$ SD over completed seeds; memory is MLX allocator peak; ± 0.00 marks bit-identical allocator peaks across seeds; D1/D8 cells are regime-dependent boundary observations discussed in the text; 95% t -CIs in Table 10).

Model	Method	Seeds	Peak mem (GiB)	Median step (s)	Tok/s	Val loss
Qwen3-0.6b (0.6B)	Full FT	3/3	5.77 $\pm$ 0.00 GiB	1.081 $\pm$ 0.003	428.6 $\pm$ 49.9	4.441 $\pm$ 0.147
Qwen3-0.6b (0.6B)	BF16 LoRA	3/3	2.92 $\pm$ 0.00 GiB	0.619 $\pm$ 0.009	809.9 $\pm$ 9.2	1.813 $\pm$ 0.002
Qwen3-1.7b (1.7B)	BF16 LoRA	3/3	5.55 $\pm$ 0.00 GiB	1.396 $\pm$ 0.005	366.2 $\pm$ 2.5	1.549 $\pm$ 0.001
Qwen3-4b (4.0B)	BF16 LoRA	1/3 (D1)	11.54 GiB	3.028	168.1	1.431
Qwen3-0.6b (0.6B)	4bit QLoRA	3/3	2.13 $\pm$ 0.00 GiB	0.697 $\pm$ 0.003	721.4 $\pm$ 4.0	1.926 $\pm$ 0.001
Qwen3-1.7b (1.7B)	4bit QLoRA	3/3	3.30 $\pm$ 0.00 GiB	1.697 $\pm$ 0.006	300.9 $\pm$ 1.1	1.634 $\pm$ 0.011
Qwen3-4b (4.0B)	4bit QLoRA	3/3	6.19 $\pm$ 0.00 GiB	3.665 $\pm$ 0.007	137.2 $\pm$ 1.1	1.472 $\pm$ 0.001
Qwen3-8b (8.2B)	4bit QLoRA	3/3	9.19 $\pm$ 0.00 GiB	10.039 $\pm$ 0.006	42.9 $\pm$ 1.1	1.377 $\pm$ 0.007
Qwen3-14b	4bit QLoRA	0/3 (D8)	—	—	—	—

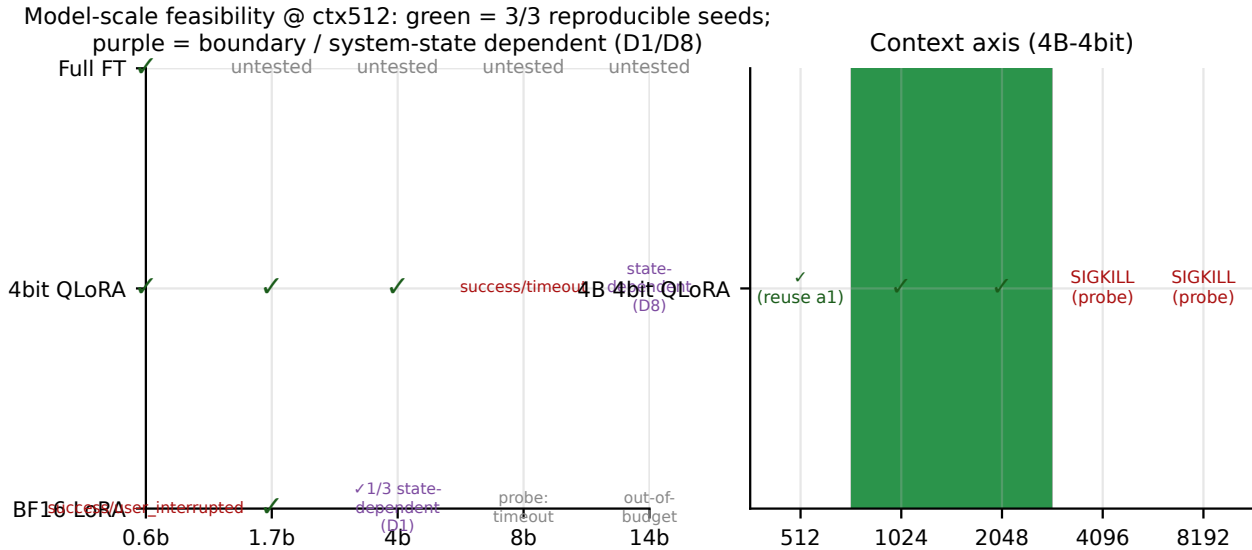


Figure 2: Feasibility map of the preregistered matrix. Left: model scale $\times$ precision at ctx512; green marks cells whose preregistered seed set completed 3/3, purple marks boundary cells whose outcome depended on system memory state (D1: 4B BF16; D8: 14B 4-bit). Right: the context axis for 4B 4-bit, with the SIGKILL failure interval shaded by evidence.

Tier-B; that label traced to an analysis-script defect (the paging metric silently failed on the flattened results table and returned infinity for all runs) and is corrected here as deviation D9. Absolute step times remain descriptive, window-specific values: the same 4B 4-bit b1 configuration measured 3.66 s/step in the 100-step axis-1 window (6.3–7.9 GiB residency, 100–122 MB/step) and 5.62 s/step in the 20-step batch-axis window (2.9–3.2 GiB residency, 231–339 MB/step; Table 4)—a 53% window drift that exceeds the $\pm 25\%$ equivalence margin, and that we therefore treat as window state, not signal. Cross-model *ratios* and paired comparisons within one window remain interpretable; absolute latencies from different windows are not compared in this paper.

4.4 Quantization Trade-offs

Across the paired scales (Figure 5, Table 3), 4-bit QLoRA reduces peak memory to $0.73\times$ (0.6B), $0.59\times$ (1.7B), and $0.54\times$ (4B) of the BF16 LoRA value ($n=7$ seed-pairs; audit H1: supported). Step-time ratios of 1.10–1.14 and 1.21–1.23 at 0.6B and 1.7B sit inside the preregistered $\pm 25\%$ equivalence margin (audit H3: supported, “no practically meaningful timing penalty under the frozen margin”). The 4B ratio of 1.21 is a single seed pair whose BF16 denominator is the zero-residency D1 rerun—a cross-window observation (Sec. 4.3)—so equivalence at 4B is descriptive rather than a within-window verdict, and we count only the two fully paired scales in the abstract. Memory ratios are window-robust: allocator peaks are

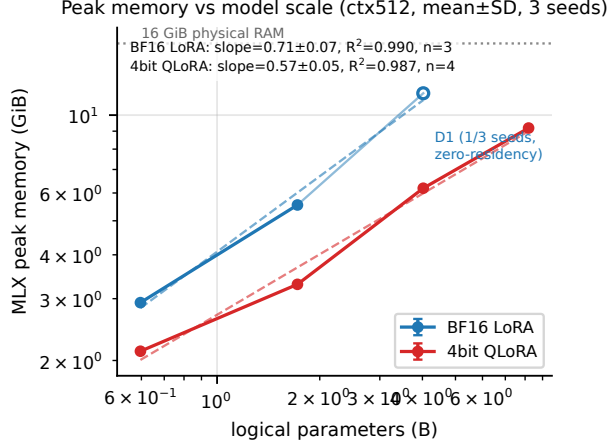


Figure 3: Peak memory vs. model scale at ctx=512 (mean±SD of three seeds; the open marker is the 4B BF16 boundary observation of D1, 1/3 seeds at zero swap residency, included in the fit). Fits are log-log least squares over the plotted points.

Table 3: Paired quantization effects (same model/seed; ratios = 4bit/BF16, mean [min–max over seed pairs]). Timing equivalence margin $\pm 25\%$ frozen before benchmark. [†]The 4B step-time ratio is a single seed pair whose BF16 denominator is the zero-residency D1 rerun, i.e. a cross-window comparison (Sec. 4.3); memory ratios are window-robust (allocator peaks are bit-stable across seeds).

Model	Mem ratio	Step-time ratio	Tok/s ratio
Qwen3-0.6b (3)	0.73	1.13 [1.10–1.14]	0.89 [0.88–0.91]
Qwen3-1.7b (3)	0.59	1.22 [1.21–1.23]	0.82 [0.82–0.83]
Qwen3-4b (1) [†]	0.54	1.21	0.81

bit-identical across seeds (± 0.00 in Table 2). Validation losses after 100 steps *differ* between precisions at both scales—0.6B: 1.93 vs. 1.81 (paired $t = 70.4$, $p = 2 \times 10^{-4}$, $n=3$ pairs); 1.7B: 1.63 vs. 1.55 (paired $t = 13.4$, $p = 0.006$)—the 0.085–0.113 absolute differences are many seed-SDs wide and the 95% CIs do not overlap. The difference is real, small in absolute terms, and points in the direction expected of 4-bit quantization noise; it is *not* evidence of training-quality equivalence, which 100-step LM loss cannot establish (Sec. 4.7).

4.5 Batch and Rank Sensitivity

Micro-batch scaling on 4-bit 4B (Table 4, Figure 6) monotonically *reduces* throughput, from 89.5 loss-bearing tokens/s at batch 1 to 66.4 at batch 2 and 34.6 at batch 4, while peak memory climbs 6.19→10.04→16.98 GiB and per-step swap-in grows

from 231–339 MB to 13.1–13.6 GB. Batch 8 is worse still: the process is killed by an external SIGKILL before it completes a step in all three seeds (exit 137, zero stdout, system-wide swap usage near 20 GiB). For the last of the three batch-8 kills the kernel’s own jetsam record is retained (Appendix C): the training process held 23.5 GiB resident (lifetime maximum 29.7 GiB) while the compressor carried 12.5 GiB and 59 MB of pages remained free—direct kernel evidence that this kill was memory exhaustion. The remaining five SIGKILL runs in the taxonomy have no retained jetsam record and are labeled by exit evidence only. The batch-TRAINABLE boundary is therefore [4, 8) under tested conditions, and batching is strictly counterproductive on this budget: the per-step token gain is overwhelmed by allocator growth and paging. All batch-axis comparisons are within one window; the 3.66 s axis-1 value for the same b1 configuration reflects a different window (Sec. 4.3). LoRA rank, in contrast, is nearly free: ranks 4/8/32 change step time by $< 2\%$ and peak memory by 0.24 GiB (Figure 7, Table 5).

4.6 Context-Length Boundary

Context length dominates every other axis (Figure 8): at 4B 4-bit, ctx1024 multiplies median step time by $6.5\times$ versus ctx512 (3.66→24.0 s) and peak memory by $1.8\times$ (6.19→11.16 GiB); ctx2048 completed all three preregistered seeds—two in the 2026-09-12 window (6.5–7.9 GiB residency) and one rerun of s123 at 2.96 GiB residency after its first attempt was SIGKILLED at 7.66 GiB (the failed attempt is the ctx2048 entry in Table 6); the allocator high-water mark reached 23.9 GiB, far beyond physical RAM, carried by macOS paging (system-wide swap usage peaked near 20 GiB during these runs). At ctx4096 and ctx8192 the process is killed by an external SIGKILL before completing a single step. The TRAINABLE context boundary is thus [2048, 4096), reported as an interval with two evidence-grade caveats: the lower edge rests on completed formal seeds, while the upper edge rests on single-seed SIGKILL probes under a synthetic long-context workload, so the interval states where failure was observed, not a measured threshold. Within Tier-B formal data the time amplification over ctx512 ($5.6\times$ at 2048) only modestly exceeds the $4\times$ token-count increase, so we report the boundary but not a dramatic time “cliff” (audit H6: partially supported, v2 rule; the preregistered-era v1 rule and its revision are disclosed as D9, App. B).

4.7 Training Effectiveness

Validation loss decreases monotonically along the recorded trajectory in every trainable run, and the

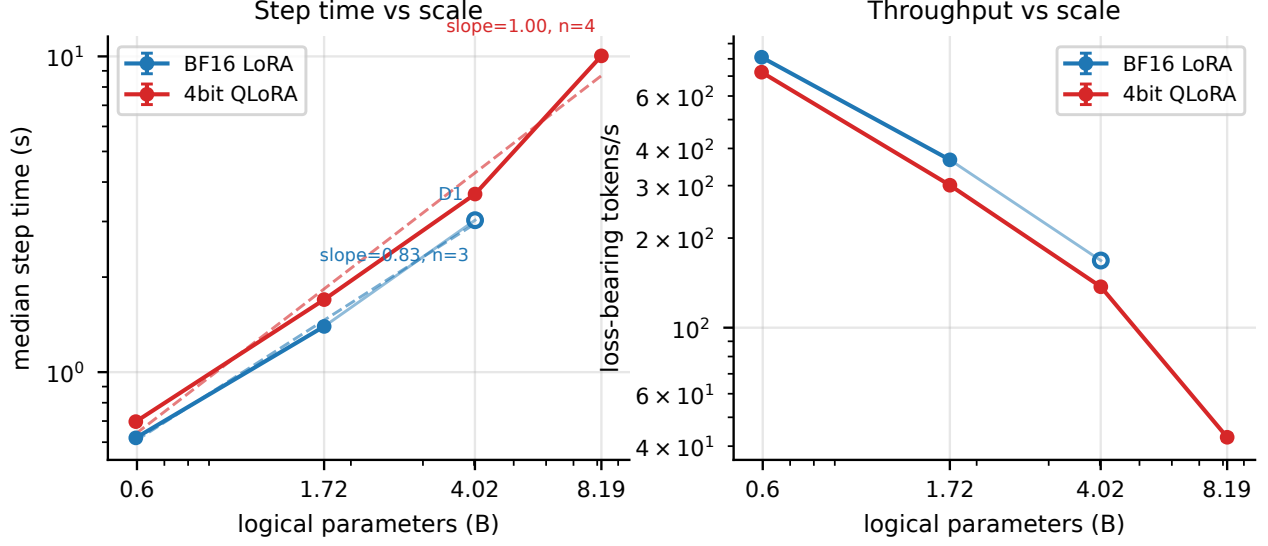


Figure 4: Training-time scaling at ctx512 (mean $\pm$ SD of 3 seeds; open marker: D1 boundary observation included in the fit). Left: median step time, with log-log fits (4-bit slope 1.00). Right: loss-bearing throughput. Absolute values are window-specific (mixed Tier-A/Tier-B stratification, Table 8); only within-window comparisons are interpreted.

Table 4: Batch axis (Qwen3-4B-4bit QLoRA, ctx512, r8, 20 steps, seeds {42,123,2026}, all cells rerun on the D5-fixed trainer in one window; mean $\pm$ SD; swap-in is the D4 paging-intensity measure, all cells Tier-B; batch-8 kills recorded zero completed steps with empty stdout).

Batch	Median step (s)	Tok/s	Peak mem (GiB)	Swap-in (MB/step)	Status
1	5.620 $\pm$ 0.055	89.5 $\pm$ 0.9	6.19 $\pm$ 0.00 GiB	231–339	success $\times$ 3
2	14.394 $\pm$ 1.730	66.4 $\pm$ 4.0	10.04 $\pm$ 0.00 GiB	1430–2329	success $\times$ 3
4	55.387 $\pm$ 4.603	34.6 $\pm$ 1.5	16.98 $\pm$ 0.00 GiB	13142–13636	success $\times$ 3
8	–	–	>16 (swap $\rightarrow$ 20 GiB)	–	SIGKILL $\times$ 3 (exit 137)

Table 5: LoRA rank axis (Qwen3-4B-4bit QLoRA, ctx512, b1, 20 steps for r4/r32; r8 point reused from the 100-step axis-1 cell per preregistration; mean $\pm$ SD; all cells Tier-B).

Rank	Median step (s)	Tok/s	Peak mem (GiB)	Swap-in
4	3.678 $\pm$ 0.010	136.8 $\pm$ 0.8	6.16 $\pm$ 0.00 GiB	383–505
8	3.665 $\pm$ 0.007	137.2 $\pm$ 1.1	6.19 $\pm$ 0.00 GiB	100–122
32	3.714 $\pm$ 0.011	134.7 $\pm$ 1.1	6.40 $\pm$ 0.00 GiB	405–527

precision difference after 100 steps is small in absolute terms (0.085–0.113) though statistically clear (Sec. Quantization Trade-offs). The divergent full-FT cell (0.6B, lr 10^{-4} , validation loss 4.44) is retained as a negative result. We make no instruction-following or converged-quality claims: the objective is LM loss on ultrachat_200k with all tokens counted over 100 steps, which supports feasibility and cost comparisons but not training-quality equivalence between precisions.

4.8 Boundary Behavior at 14B

We report the 14B 4-bit cell as an explicit boundary case study (D8): a configuration observed to enter training under favorable conditions, but not reproducibly sustained under the formal workload.

Observation Three raw records constitute the evidence. (1) A 20-step probe at 2.5 GiB swap residency loaded and trained at 0.68 s/step (**success**). (2) A formal attempt launched at 6.6 GiB residency completed zero steps in 24 minutes before the operator interrupted it (KeyboardInterrupt; recorded under an unresolved-failure terminal state, D8). (3) A second formal attempt at 3.5 GiB residency exhibited $\sim$ 40 minutes of model loading, then progressed to 70 of 100 steps (70 recorded step lines in the raw stdout log), with step times of 9.4–105 s that peaked mid-run and partially recovered, before stalling for the final $\sim$ 23 minutes until the 7,200 s timeout killed it. Residency figures are the pre-run `initial_swap_bytes` of each raw record.

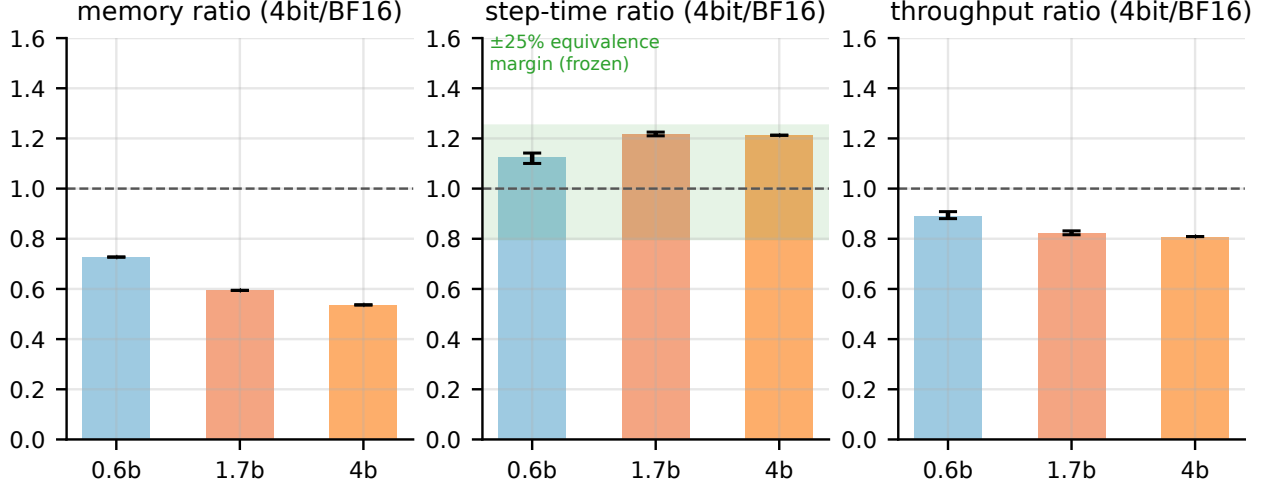


Figure 5: Seed-paired 4-bit QLoRA / BF16 LoRA effects (ratios per matched seed; bars: mean, whiskers: min-max over seed pairs). The step-time panel shows the preregistered $\pm 25\%$ equivalence margin; all three scales sit inside it.

All three records are retained unaltered; two further attempts were never executed (D8).

Interpretation These observations are consistent with 14B crossing from reproducible training into a regime where system memory residency and paging state materially affect execution: the same regime dependence documented for 4B BF16 (D1), reached earlier in the scale ladder. This is an interpretation, not a direct measurement of untrainability: we did not observe allocator exhaustion, and the probe demonstrates that favorable conditions exist.

Limitation We did not complete a three-seed formal series at 14B, did not attempt to manufacture a low-residency window (no reboots or system-state manipulation to chase success, per D8), and therefore can neither claim that 14B is reproducibly trainable nor rule it out on 16 GiB; the 7,200s limit that terminated the second attempt is the supervisor-wide configured bound (2,400–7,200s across formal configs, Table 6), not a per-cell tuned value, and its sensitivity is untested. The preregistered 14B ctx2048 probe was cancelled without execution. The honest summary is: under the tested formal workload and observed system states, 14B 4-bit QLoRA was not sustained to completion, while a favorable-state probe did complete.

5 Failure Analysis

Of 118 finalized runs, 27 are failures; all are retained, and none is reclassified as OOM without kernel evi-

dence. The taxonomy (Table 6) splits into: external SIGKILLS under sampled memory pressure (batch-8 runs, ctx2048 first attempt, ctx4096/8192 probes: exit 137 with zero or partial output; one kill kernel-verified, App. C); timeouts under swap residency (14B formal, 8B BF16 probe, and the 8B first formal attempt); operator interrupts now documented as system-state evidence (D1’s 27-minute zero-step 4B BF16 run; the 14B 24-minute halt); and nine formal attempts invalidated by a trainer implementation bug (D5) whose variable-length validation batching crashed for microbatch ≥ 2 . These nine are excluded from aggregation by an error-signature rule, rerun in full on the fixed trainer (12/12 valid outcomes), and preserved immutably. A coverage audit reconciles all 118 raw runs against the processed table with machine-readable dispositions (46 aggregated successes, 14 duplicates, 19 superseded, 9 implementation-invalid, 9 failed, 21 non-formal); all 46 aggregated runs pass SHA-256 manifest re-verification, while for six runs the manifest digest no longer matches the finalized logs—manifest files are present, but the supervisor appended to stdout/stderr after writing the manifest, invalidating the digest (five legacy interrupted/timeout runs, D6, plus the 2026-09-15 14B timeout; zero aggregation impact).

6 Discussion

6.1 What Does 16 GB Actually Enable?

The results separate into three tiers once the frozen thresholds are applied honestly (Table 9). Through

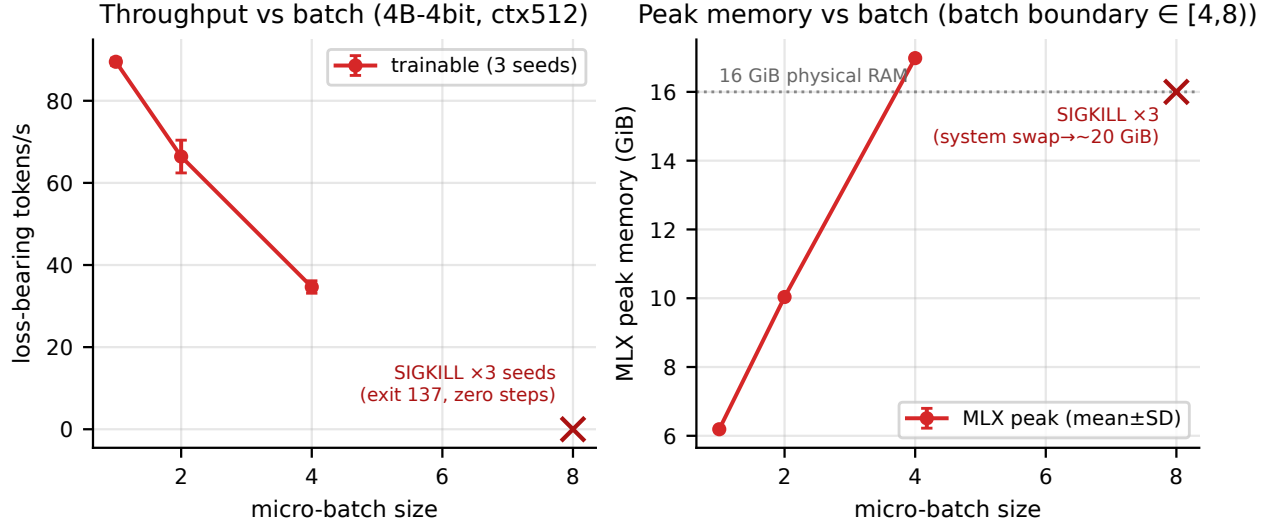


Figure 6: Micro-batch axis (4B 4-bit, ctx512): throughput falls and peak memory rises with batch size; batch 8 is SIGKILLED in all seeds (boundary $\in [4, 8)$).

4B, 4-bit QLoRA is TRAINABLE and PRACTICAL on P2 (median step ≤ 10 s) with stable three-seed reproduction. At 8B, 4-bit remains TRAINABLE but misses the frozen P2 threshold by 0.039s (10.039 vs. 10s); under the $\pm 2\times$ sensitivity it is PRACTICAL at a 20s threshold and fails at 5s, so we call it borderline-PRACTICAL rather than blur the boundary. Its 42.9 tokens/s also fails the EFFICIENT criterion, but that number is a Tier-B absolute value under the resident paging state (Sec. 4.3); a favorable window could move it, which is why EFFICIENT was preregistered as exploratory. The P1 swap-growth criterion is system-wide and is confounded by background load: measured swap growth was 5.2 GiB on the 0.6B cell but 3.1–3.7 GiB on the 4B/8B cells (per-run peak-minus-initial swap in the processed data; the resulting P1 pass/fail pattern is in Table 9)—an inversion that tracks background processes, not the training working set, so we do not gate PRACTICAL on P1 alone. At 14B, behavior becomes system-state-dependent: favorable conditions admit real training steps, while the formal workload was not sustained under observed residencies. BF16 LoRA is PRACTICAL through 1.7B. The binding constraints are, in order, context length, micro-batch, then model scale, inverting the usual intuition that parameters come first. The most valuable single optimization for this class of hardware would attack the fixed non-weight memory overhead that the affine memory fit isolates (~ 1.8 GiB intercept).

6.2 Loadable vs Trainable vs Practical, and How to Report It

The three-layer vocabulary is not academic: every “boundary” in this benchmark moved with system state. The same 4B BF16 configuration is TRAINABLE in 451s at zero swap residency and cannot finish one step in 27 minutes at 6.1–6.3 GiB residency (D1); 14B 4-bit spans a 0.68s/step probe and a 70/100-step timeout (D7/D8); and 8B itself, our “reproducible” boundary cell, timed out at 16.4 GiB residency before completing 3/3 in a 3.6–4.8 GiB window (D7). The favorable and unfavorable readings are the same kind of evidence, which is the point. Feasibility claims that do not report swap residency are therefore not reproducible in kind.

We recommend memory-residency state become a mandatory field in consumer-hardware training reports. Concretely, the field should cover (i) system-wide swap before/after and the 1Hz sampled peak, (ii) the before/after `vm.stat` snapshot including compressor occupancy and swap-in/out counters (already recorded by our supervisor; per-step paging intensity is derivable, Table 8), and (iii) per-process resident footprint where available. The analogy to clock state in GPU benchmarks has a stated limit: clock state is a device-local, standardized quantity, whereas system-wide swap is a machine-history property shared with every other process, so residency numbers are comparable across reports only alongside the sampling protocol, not as bare values. The complementary route is *controlling* state—fresh-boot windows—which we deliberately declined for the 14B cell (D8) to avoid a

Table 6: Failure taxonomy over all 27 failed runs (terminal states preserved verbatim; categories are evidence-based labels, never reclassifications).

Category	Runs	Configurations	Evidence
External SIGKILL (memory pressure)	6	qwen3-4b-4bit (b8×3, ctx2048, ctx4096 [probe], ctx8192 [probe])	exit 137 / SIGKILL, steps not recorded; kernel JetsamEvent confirms one b8 kill (App. C); rest SIGKILL-consistent only, not reclassified as OOM; terminal state: runtime_error (SIGKILL)
Supervisor timeout	4	qwen3-14b-4bit; qwen3-4b (BF16); qwen3-8b (BF16) [probe]; qwen3-8b-4bit	killed at the configured supervisor limit 2400–7200s under multi-GiB swap residency; terminal state: timeout (SIGKILL)
Operator interrupt	5	qwen3-0.6b (BF16); qwen3-4b (BF16)×2; qwen3-4b-4bit×2	operator halt after 15–1627s; retained as system-state evidence (D1/D7); terminal state: user_interrupted
Trainer implementation bug (D5)	9	qwen3-4b-4bit (b2×3, b4×3, b8×3)	python traceback in variable-length validation batching (micro-batch ≥ 2); rerun on the fixed trainer; terminal state: runtime_error
Pre-formal pipeline debug	2	qwen3-0.6b (BF16)×2	python traceback before the formal matrix was frozen; terminal state: runtime_error
Unresolved failure	1	qwen3-14b-4bit	no specific pattern in exit evidence; 14B attempt attributed to operator KeyboardInterrupt per D8; terminal state: unknown_failure

special-system-state confounder; both routes belong in a reporting standard.

6.3 Implications for Consumer LLM Research

Three practical rules follow. (1) Quantization pays twice on unified memory: it buys headroom ($0.54\text{--}0.73\times$ memory) at a time cost measured at $1.10\text{--}1.23\times$ and bounded inside the frozen $\pm 25\%$ margin, and extends reach by at least two model sizes. (2) Do not batch: on a paging-capable unified budget, throughput falls monotonically with micro-batch and the hard failure at batch 8 arrives abruptly (one kill kernel-verified, App. C). (3) Treat context as the primary resource: the [2048,4096] boundary with the allocator peak at 23.9 GiB (vs. 16 GiB physical) shows the system will carry the excess via swap until the process dies: “it started” says nothing about “it will finish”.

6.4 Limitations

Single machine, single SoC generation (M4, 16 GiB); all conclusions are single-point observations on that machine. Absolute times are window-specific under a mixed Tier-A/Tier-B picture (Table 8), so we compare only within-window ratios. The 8B “reproducible” cell is three-of-three within one favorable window (first attempt at high residency timed out, D7); its success-rate interval is wide (Clopper–Pearson lower bound 0.29). Rerun decisions were not uniformly preregistered: first attempts that failed for environment-attributable reasons (8B s42, ctx2048 s123) were rerun under the D7 information-gain rule with both outcomes retained, while the 4B BF16 cell was frozen as boundary evi-

dence (D1) and 14B attempts were stopped (D8)—we state this asymmetry rather than smooth it over. The 14B “not sustained” verdict is bounded by the 7,200s supervisor timeout, whose sensitivity we did not measure. LM-loss-only objectives with no prompt masking; 20-step runs on the context/batch/rank axes limit distributional detail (54 measured steps per cell after warm-up exclusion, 3 seeds $\times$ 18); system-level swap counters include background processes; the context upper edge rests on single-seed probes. Operational variables that matter to end users—thermal and fan behavior under sustained GPU load, SSD write wear from multi-GiB swap traffic, disk-space budgeting, and wall-clock extrapolation to full training runs—are outside this benchmark’s scope. Deviations D1–D9 are summarized in Appendix B and reported in full in the repository.

6.5 Ethics and Licensing

The benchmark targets publicly released model weights (Apache 2.0 for Qwen3) and a publicly released dataset (ultrachat_200k, MIT license); no synthetic workload contributes to quality claims. All measurements were performed on the authors’ own hardware.

7 Conclusion

On this 16 GiB unified-memory Mac, 4-bit QLoRA is the default answer for fine-tuning: it roughly halves memory ($0.54\text{--}0.73\times$) at a time cost of $1.10\text{--}1.23\times$ that stays inside the frozen $\pm 25\%$ margin, completes the preregistered seed set at 8B within one favorable residency window, and leaves rank choice nearly

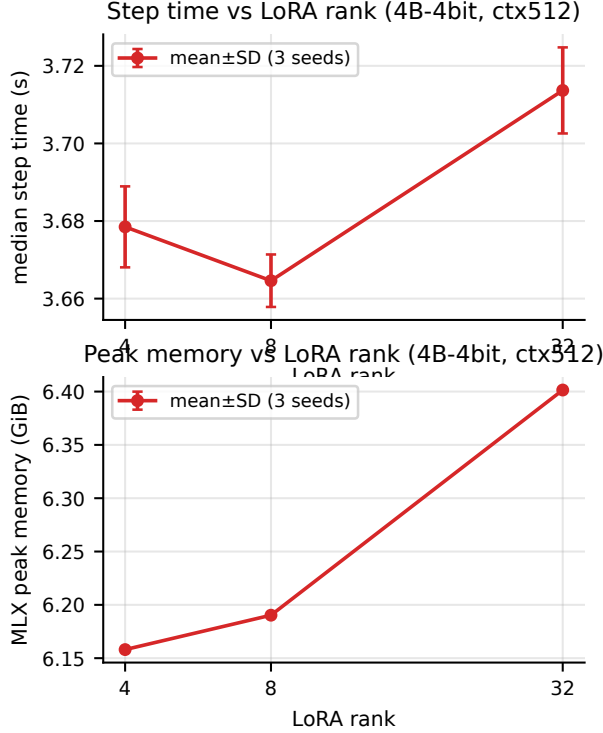


Figure 7: LoRA rank axis (4B 4-bit, ctx512, b1; r8 reuses the axis-1 cell per preregistration): neither step time nor peak memory moves materially across ranks 4–32.

free. The boundaries that actually bind are context ($\in [2048, 4096]$ tokens) and micro-batch ($\in [4, 8]$, with monotonically negative throughput returns). Both boundaries, like the 4B BF16, 8B, and 14B cells, shift with the machine’s swap residency, making system-state reporting a prerequisite for any reproducible consumer-hardware training claim. Every number above is regenerated from immutable raw runs by committed scripts; the failures are part of the result.

References

- [1] Oluwaseun A. Ajayi and Ogundepo Odunayo. Benchmarking on-device machine learning on Apple Silicon with MLX. In *6th Deep Learning Indaba (DLI 2024)*, Dakar, Senegal, 2025. Non-archival presentation; arXiv:2510.18921.
- [2] Apple Inc. Apple introduces M4 chip. Press release, 2024.
- [3] Gaurab Baral, Aaditya Khanal, Yangyang Tao, and Junxiu Zhou. Knowledge distillation from large reasoning models to compact student models: A case study on the John O’Bryan mathematics competition. *arXiv preprint arXiv:2606.31048*, 2026.
- [4] Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. Training deep nets with sublinear memory cost. *arXiv preprint arXiv:1604.06174*, 2016.
- [5] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. LLM.int8(): 8-bit matrix multiplication for transformers at scale. In *Advances in Neural Information Processing Systems*, volume 35, 2022.
- [6] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- [7] Ning Ding, Yulin Chen, Bokai Xu, Yujia Qin, Shengding Hu, Zhiyuan Liu, Maosong Sun, and Bowen Zhou. Enhancing chat language models by scaling high-quality instructional conversations. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 3029–3051, 2023. arXiv:2305.14233; camera-ready author list per ACL Anthology.
- [8] Dahua Feng, Zhiming Xu, Rongxiang Wang, and Felix Xiaozhu Lin. Profiling Apple Silicon performance for ML training. *arXiv preprint arXiv:2501.14925*, 2025.
- [9] Jiaxiang Geng, Lunyu Zhao, Yiyi Lu, and Bing Luo. MobileFineTuner: A mobile-native framework for on-device LLM fine-tuning in real-world embedded AI applications. *arXiv preprint arXiv:2512.08211*, 2025.
- [10] Georgi Gerganov and contributors. llama.cpp: LLM inference in C/C++. GitHub repository, 2023.
- [11] Awni Hannun, Jagrit Digani, Angelos Katharopoulos, and Ronan Collobert. MLX: Efficient and flexible machine learning on apple silicon. GitHub repository, 2023.
- [12] Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin de Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. Parameter-efficient transfer learning for NLP. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, pages 2790–2799, 2019.
- [13] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang,

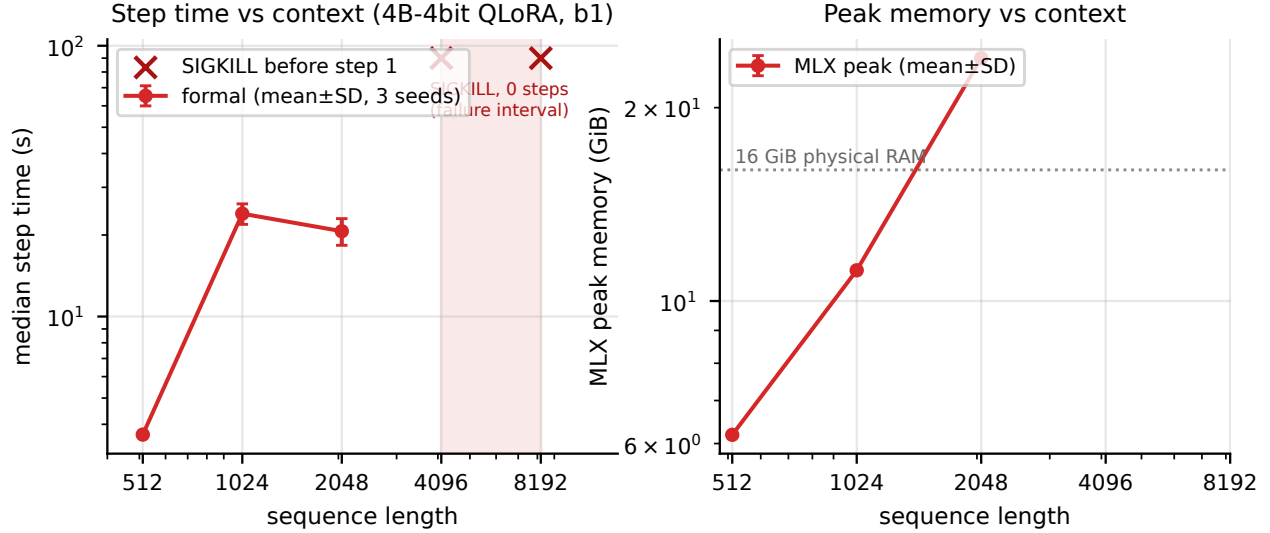


Figure 8: Context scaling for 4B 4-bit QLoRA: step time (left) and peak memory (right). Crosses mark SIGKILL probe failures; shaded region marks the failure interval.

- Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.
- [14] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.
- [15] Kai Lv, Yuqing Yang, Tengxiao Liu, Qipeng Guo, and Xipeng Qiu. Full parameter fine-tuning for large language models with limited resources. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL): System Demonstrations*, pages 8187–8198, 2024. Camera-ready author list per ACL Anthology; arXiv:2306.09782.
- [16] MLX team. mlx-lm: LLM fine-tuning and inference with MLX. GitHub repository, 2023.
- [17] Varun Rajesh, Om Jodhpurkar, Pooja Anbuselvan, Mantinder Singh, Ashok Jallepali, Shantanu Godbole, Pradeep Kumar Sharma, and Hritvik Shrivastava. Production-grade local LLM inference on Apple Silicon: A comparative study of MLX, MLC-LLM, Ollama, llama.cpp, and PyTorch MPS. *arXiv preprint arXiv:2511.05502*, 2025.
- [18] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [19] Jiawei Zhao, Zhenyu Zhang, Beidi Chen, Zhangyang Wang, Anima Anandkumar, and Yuandong Tian. GaLore: Memory-efficient LLM training by gradient low-rank projection. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.
- [20] Yaowei Zheng, Richong Zhang, Junhao Zhang, Yanhan Ye, and Zheyuan Luo. LlamaFactory: Unified efficient fine-tuning of 100+ language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL): System Demonstrations*, pages 400–410, 2024. Camera-ready author list per ACL Anthology; arXiv:2403.13372.

A Reproducibility Instructions

The repository ships the complete chain:

1. `uv sync` recreates the locked Python environment.
2. `uv run python scripts/run_experiment.py --config <yaml> --` launches any experiment under the supervisor; formal configurations are `configs/experiments/formal/`.
3. `data/formal.sft.v1/` contains the frozen dataset with `MANIFEST.json` and `SHA256SUMS`.
4. `uv run python scripts/run_analysis.py` regenerates every processed table, figure, and summary from `results/raw/`.

5. Every raw result directory contains its exact command, effective configuration, environment snapshot, logs, and a SHA-256 manifest.

B Preregistration Deviations

The preregistration was frozen in commit `3dc33d2` (2026-09-12, before any formal run). All deviations are logged in `research/deviations.md` with raw-evidence paths; one-line summaries:

Table 7: Preregistration deviations D1–D9 (full records in `research/deviations.md`).

ID	Summary
D1	4B BF16 cell demoted to boundary observation after 27-min zero-step run at 6+ GiB residency; a post-reboot zero-residency rerun succeeded (451 s), establishing regime dependence.
D2	A runner config-hash mismatch duplicated some 0.6B/1.7B groups (up to 3×); duplicates retained in raw, deduplicated in analysis (earliest retained success per (group, seed)).
D3	Swap gate strengthened after a 14B attempt stalled 24 minutes under multi-GiB residency: formal runs require pre-run system swap ≤ 8.5 GiB.
D4	Paging stratification frozen before the axis-1 rerun: Tier-A <50 MB swap-in/step (compute-bound) vs. Tier-B; timing comparisons restricted to within-window.
D5	Trainer bug (variable-length validation batching, crashed for micro-batch ≥ 2): 9 runs invalidated, full batch axis rerun on the fixed trainer (12/12 valid outcomes); historical raw preserved.
D6	Post-finalize log appends invalidated the manifest digest for five interrupted/timeout runs (manifest files present; evidence quality downgraded, zero aggregation impact—all 46 aggregated runs re-verify).
D7	14B formal timeout reattributed to environment; 8B/ctx2048 first-attempt failures rerun under an information-gain rule with both outcomes retained.
D8	Stopping decision: 14B reported as a system-state-dependent boundary case; no low-residency windows were manufactured.
D9	Analysis-layer deviations disclosed post-audit: (a) H2/H6 audit rules were revised after results landed (H2: v1 anchored on 14B 3/3, revised to anchor on 8B after the D7 timeout—under v1 the H2 verdict would read <i>not supported</i> ; H6 likewise runs a v2 dual-evidence rule); (b) the paging metric failed silently on the flattened table, labeling every run Tier-B—corrected, giving the mixed stratification of Table 8.

C Kernel Jetsam Evidence for One Batch-8 Kill

During revision we retrieved the kernel-generated jetsam report `JetsamEvent-2026-09-16-004948.ips` (bug-type 298) from `/Library/Logs/DiagnosticReports/`; a copy with SHA-256 sidecar is preserved at

`research/evidence/`. Its timestamp (2026-09-16 00:49:48 local) falls inside the third batch-8 run’s execution window (started 00:45:51). The report names `python3.13` (the training process) as the largest process with 1,539,055 resident pages (≈ 23.5 GiB at 16 KiB pages; lifetime maximum ≈ 29.7 GiB), while the memory compressor held 12.5 GiB and only 59 MiB of pages remained free, with 24 processes jettisoned for compressor-space shortage. This is direct kernel evidence that this kill was memory exhaustion. No jetsam reports were retained for the other five external-SIGKILL runs; their classification rests on exit-137 evidence and sampled swap levels, and is labeled accordingly in Table 6.

D Paging Stratification, Threshold Sensitivity, and CIs

Table 8 reports the per-group D4 paging stratification computed from the frozen `vm.stat` snapshots; Table 9 the $\pm 2\times$ threshold sensitivity promised in the preregistration; Table 10 the axis-1 cells with 95% *t*-CIs. The 4-bit log-log memory slope has 95% CI $[0.37, 0.78]$ (df 2); the step-time slope $[0.62, 1.37]$ (df 2); the three-point BF16 memory slope CI $[-0.22, 1.65]$ is reported as carrying no inferential weight. A power law and an affine (peak $\approx w\cdot\text{params} + c$) model fit the 4-bit points comparably ($\Delta\text{AIC } 3.2$; both $R^2 > 0.98$); the affine intercepts (~ 1.8 GiB 4-bit, ~ 1.3 GiB BF16) quantify the fixed non-weight component.

E Measurement Validation

Summary of `docs/measurement_validation.md`: (A) step timing requires the explicit `mx.eval` synchronization present in the trainer; asynchronous dispatch under-reports step time by $\sim 4,500\times$ on a reference workload. (B) `get_peak_memory` counts MLX allocator buffers in bytes; a 1 GiB allocation increases both active and peak memory by exactly 1 GiB after reset. (C) All raw results are arithmetically consistent with their per-step timing artifacts under their declared interval definitions. (D) The 1 Hz swap sampler consumes 1.47% of one CPU core at $5\times$ the formal cadence.

Table 8: Per-group paging stratification for the aggregated groups (D4 rule, computed from the frozen `vm_stat` snapshots in each raw record): Tier-A = swap-in <50 MB/step (compute-bound), Tier-B otherwise. Residency is the pre-run system swap level. The axis1b 300-step duplicate-retention group (D2) is excluded from the aggregated matrix and hence here.

Group	Runs	Tier-A	Tier-B	Swap-in (MB/step)	Residency (GiB)
a1: 0.6b-4bit-qlora	3	3	0	8–10	6.3–6.4
a1: 0.6b-bf16-full	3	0	3	60–98	6.5–7.5
a1: 0.6b-bf16-lora	3	3	0	11–19	6.3–6.4
a1: 1.7b-4bit-qlora	3	3	0	21–34	6.4–7.9
a1: 1.7b-bf16-lora	3	0	3	59–82	6.3–6.5
a1: 4b-4bit-qlora	3	0	3	100–122	6.3–7.9
a1: 4b-bf16-lora	1	1	0	22–22	0.0–0.0
a1: 8b-4bit-qlora	3	0	3	732–848	3.6–4.8
a2: ctx1024	3	0	3	7474–7957	6.2–8.0
a2: ctx2048	3	0	3	7838–8196	3.0–7.9
a3: r32	3	0	3	405–527	7.8–7.9
a3: r4	3	0	3	383–505	7.7–8.0
a4: b1	3	0	3	231–339	2.9–3.2
a4: b2	3	0	3	1430–2329	3.1–3.5
a4: b4	3	0	3	13142–13636	3.1–3.2

Table 9: Threshold sensitivity (preregistration Sect. 5: *results must be shown if conclusions flip under $\pm 2\times$ threshold changes*). Pass/fail of the frozen PRACTICAL criteria P1 (swap growth ≤ 4 GiB), P2 (median step ≤ 10 s) and the exploratory EFFICIENT criterion (≥ 100 loss-bearing tokens/s) at $0.5\times/1\times/2\times$ the frozen values.

Cell	P1 (2/4/8 GiB)	P2 (5/10/20 s)	Eff. (50/100/200)
0.6b 4bit	n/n/Y	Y/Y/Y	Y/Y/Y
1.7b 4bit	n/Y/Y	Y/Y/Y	Y/Y/Y
4b 4bit	n/Y/Y	Y/Y/Y	Y/Y/n
8b 4bit	n/Y/Y	n/n/Y	n/n/n
0.6b BF16	n/n/Y	Y/Y/Y	Y/Y/Y
1.7b BF16	n/Y/Y	Y/Y/Y	Y/Y/Y
4b BF16	n/Y/Y	Y/Y/Y	Y/Y/n

Table 10: Axis-1 cells, mean [95% t -CI] (preregistration Sect. 9: wide intervals reported honestly; CI = $t_{0.975, n-1}$ SD/ $\sqrt{n}$; per-seed SD in Table 2).

Cell	Median step (s)	Tok/s	Val loss
0.6b 4bit	0.70 [0.69,0.71]	721 [711,731]	1.93 [1.92,1.93]
1.7b 4bit	1.70 [1.68,1.71]	301 [298,304]	1.63 [1.61,1.66]
4b 4bit	3.66 [3.65,3.68]	137 [135,140]	1.47 [1.47,1.47]
8b 4bit	10.04 [10.02,10.05]	43 [40,46]	1.38 [1.36,1.39]
0.6b BF16	0.62 [0.60,0.64]	810 [787,833]	1.81 [1.81,1.82]
1.7b BF16	1.40 [1.38,1.41]	366 [360,372]	1.55 [1.55,1.55]
0.6b full1FT	1.08 [1.07,1.09]	429 [305,553]	4.44 [4.08,4.81]